Test cases and Exception Handling

Simple Arithmetic Calculator

```java
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public double divide(double a, double b) {
        if (b == 0) {
            throw new ArithmeticException("Cannot divide by zero");
        }
        return a / b;
    }

    public double power(double a, double b) {
```

Test Cases

```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class CalculatorTest {

    Calculator calc = new Calculator();

    @Test
    void testAdd() {
        assertEquals(5, calc.add(2, 3));
    }

    @Test
    void testSubtract() {
        assertEquals(6, calc.subtract(10, 4));
    }

    @Test
    void testMultiply() {
        assertEquals(21, calc.multiply(3, 7));
    }

    @Test
    void testDivide() {
        assertEquals(4.0, calc.divide(12, 3));
```

Test Cases

```java
    @Test
    void testDivide() {
        assertEquals(4.0, calc.divide(12, 3));
    }

    @Test
    void testDivideByZero() {
        assertThrows(ArithmeticException.class, () -> calc.divide(5, 0));
    }

    @Test
    void testPower() {
        assertEquals(8.0, calc.power(2, 3));
    }

    @Test
    void testNegativeNumbers() {
        assertEquals(-2, calc.add(-5, 3));
        assertEquals(-15, calc.multiply(-5, 3));
    }

    @Test
    void testFloatingPointPrecision() {
        assertEquals(0.3, calc.add(0.1, 0.2), 1e-9);
    }
}
```

Pizza Shop Example

The pizza shop implementation looks very similar:

```
public class PizzaShop extends Shop implements Nameable {

    String companyName;

    String[] foodOfferings = {
        "Pizza",
        "Spaghetti",
        "Garden Salad",
        "Anitpasto",
        "Calzone"
    }

    public String[] getInventory() {
```

Pizza Shop Example

```
        return foodOfferings;

    }

    public void buyInventory(String item) {

        System.out.println("\nYou have just purchased " + item);

    }

    public String getName(){

        return companyName;
    }

    public void setName(String name){

        companyName = name;
    }

}
```

Test cases

```java
class PizzaShopTest {

    private PizzaShop shop;

    @BeforeEach
    void setUp() {
        shop = new PizzaShop();
    }

    @Test
    void testGetInventory() {
        String[] expected = {
            "Pizza",
            "Spaghetti",
            "Garden Salad",
            "Antipasto",
            "Calzone"
        };

        assertArrayEquals(expected, shop.getInventory());
    }
```

Test cases

```java
    @Test
    void testSetNameAndGetName() {
        shop.setName("Mario's Pizza");
        assertEquals("Mario's Pizza", shop.getName());
    }

    @Test
    void testBuyInventoryPrintsMessage() {
        // Capture System.out
        java.io.ByteArrayOutputStream outContent = new java.io.ByteArrayOutputStream()
        System.setOut(new java.io.PrintStream(outContent));

        shop.buyInventory("Pizza");

        String output = outContent.toString().trim();
        assertTrue(output.contains("You have just purchased Pizza"));
    }
}
```

Exception Handling

Basic Try–Catch Example

```java
try {
    int x = 10 / 0;  // will cause ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero!");
}
```

Try–Catch–Finally

```java
try {
    String name = null;
    System.out.println(name.length());
} catch (NullPointerException e) {
    System.out.println("Name cannot be null!");
} finally {
    System.out.println("Execution finished.");
}
```

Multiple Catch Blocks

```java
try {
    int[] arr = {1, 2, 3};
    System.out.println(arr[5]);  // ArrayIndexOutOfBounds
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Invalid array index!");
} catch (Exception e) {
    System.out.println("General error occurred.");
}
```

Catching Multiple Exceptions in One Block

```java
try {
    int num = Integer.parseInt("abc");
} catch (NumberFormatException | NullPointerException e) {
    System.out.println("Invalid number format!");
}
```

Throwing an Exception Manually

```java
public void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("Age cannot be negative!");
    }
}
```

Creating a Custom Exception

```
class InvalidPizzaOrderException extends Exception {
    public InvalidPizzaOrderException(String message) {
        super(message);
    }
}
```

Throwing it:

```java
void orderPizza(int quantity) throws InvalidPizzaOrderException {
    if (quantity <= 0) {
        throw new InvalidPizzaOrderException("Quantity must be positive!");
    }
}
```

Catching it:

```java
try {
    orderPizza(0);
} catch (InvalidPizzaOrderException e) {
    System.out.println(e.getMessage());
}
```